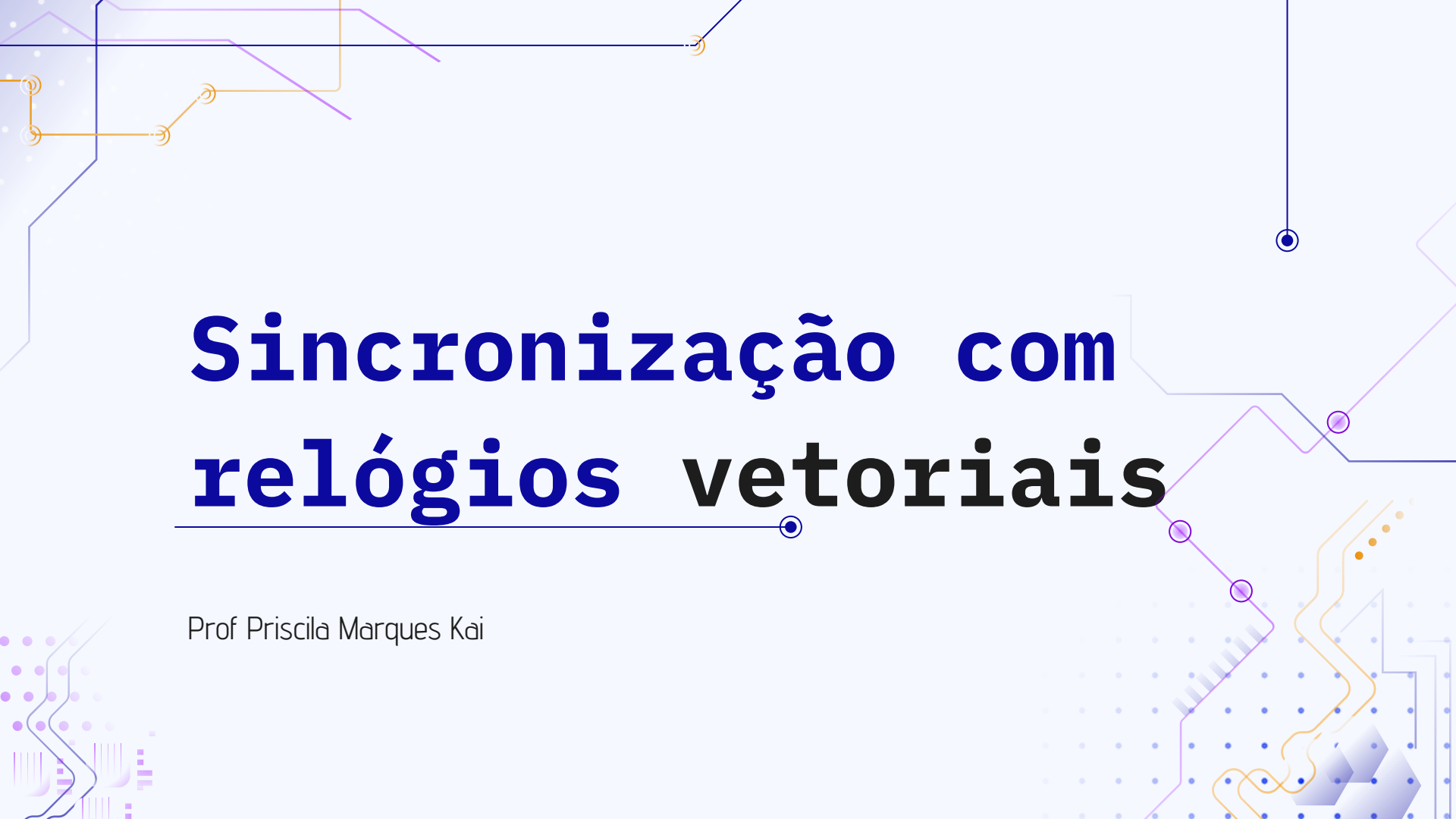


The background features a light blue gradient with abstract circuit-like patterns. Purple and orange lines, some straight and some curved, crisscross the slide. Small circles, some solid and some hollow, are placed at various points along these lines. In the bottom right corner, there is a cluster of blue dots and a series of blue arrows pointing towards the right.

Sincronização com relógios vetoriais

Prof Priscila Marques Kai

Tempo Lógico

A noção de **tempo lógico** parte da ideia de que, em sistemas distribuídos, o que realmente importa são os **eventos** e **suas relações de ocorrência**, e não a passagem do tempo físico. Isso ocorre porque o tempo é **relativo a cada processo**.

Nesse contexto, surgem os **relógios lógicos**, que “ticam” sempre que ocorre um **evento significativo**, em vez de avançarem em intervalos fixos de tempo real.

Tempo Lógico

Para compreender os relógios lógicos, é fundamental entender a **relação *happened-before***, proposta por **Leslie Lamport**.

- utilizado para descrever a **ordem dos eventos** em sistemas computacionais, especialmente em **sistemas distribuídos**.

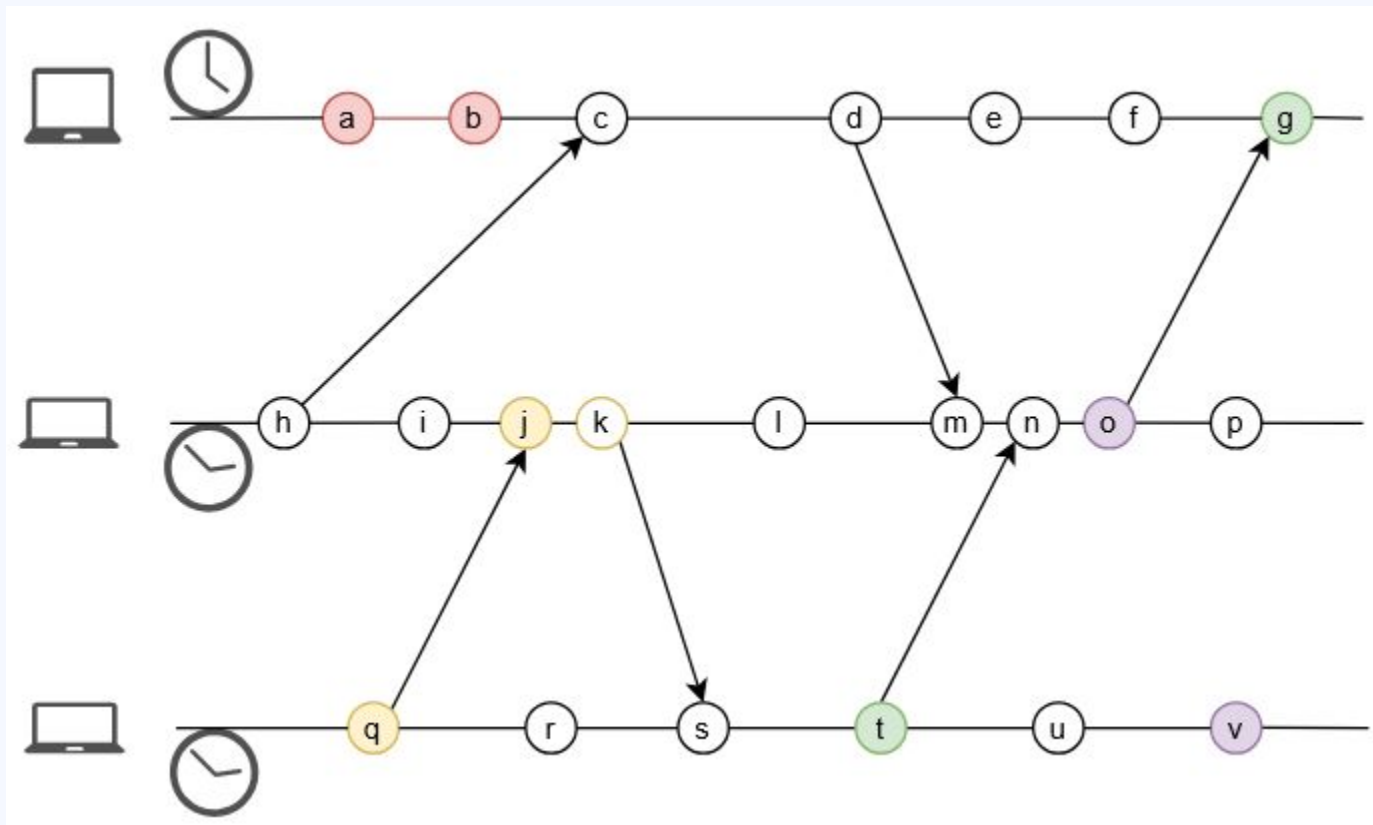
Tempo Lógico

Relação Happened-Before: expressa a **causalidade entre eventos**.

$$a \rightarrow b$$

- se um evento **a** ocorreu antes de outro evento **b**, **a precede b** se uma das seguintes condições é verdadeira:
 1. **Ordem local:** os eventos **a** e **b** pertencem ao mesmo processo (ou *thread*), e **a** foi executado **antes** de **b**.
 2. **Comunicação entre processos:** **a** é o **envio** de uma mensagem por um processo e **b** é a **recepção** dessa mesma mensagem por outro processo.
 3. **Transitividade:** existe um evento **c** tal que $a \rightarrow c$ e $c \rightarrow b$.

Tempo Lógico



Tempo Lógico

Essa relação fornece a base para a construção de **relógios lógicos**, pois permite representar a ordem causal dos eventos **sem depender do tempo físico** — apenas das **relações de dependência e comunicação** entre os processos.

Por que capturar a causalidade entre eventos?

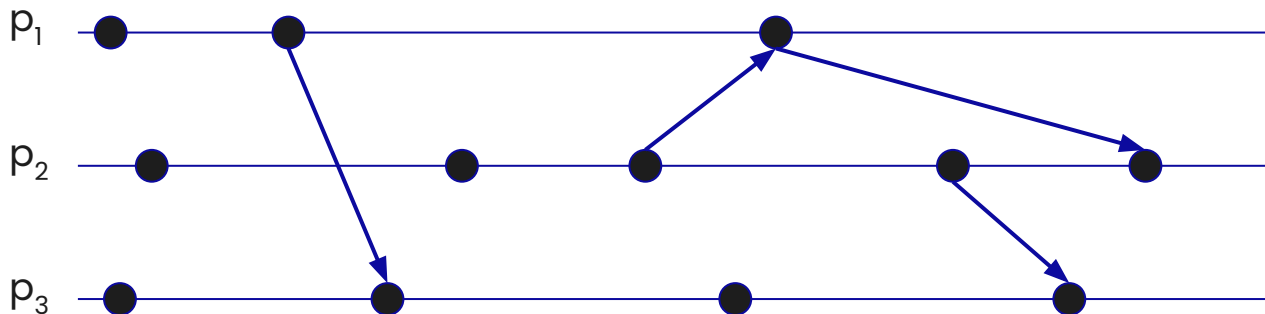
Identificar quais eventos **causam** ou **dependem** de outros possibilitam **ordenar o processamento** de modo que o sistema mantenha uma **lógica coerente de execução**.

Exemplo: sistema de **e-mail distribuído**

- Um usuário não deve receber primeiro a **resposta** e só depois recebe a **mensagem original A**.
- Para que a comunicação faça sentido, deve-se **adiar** a leitura da mensagem até que **A** tenha sido recebida e processada.

Relógios lógicos de Lamport - exemplo

- Enumere os eventos de cada processo de acordo com o que aprendido em Relógios Lógicos de Lamport.

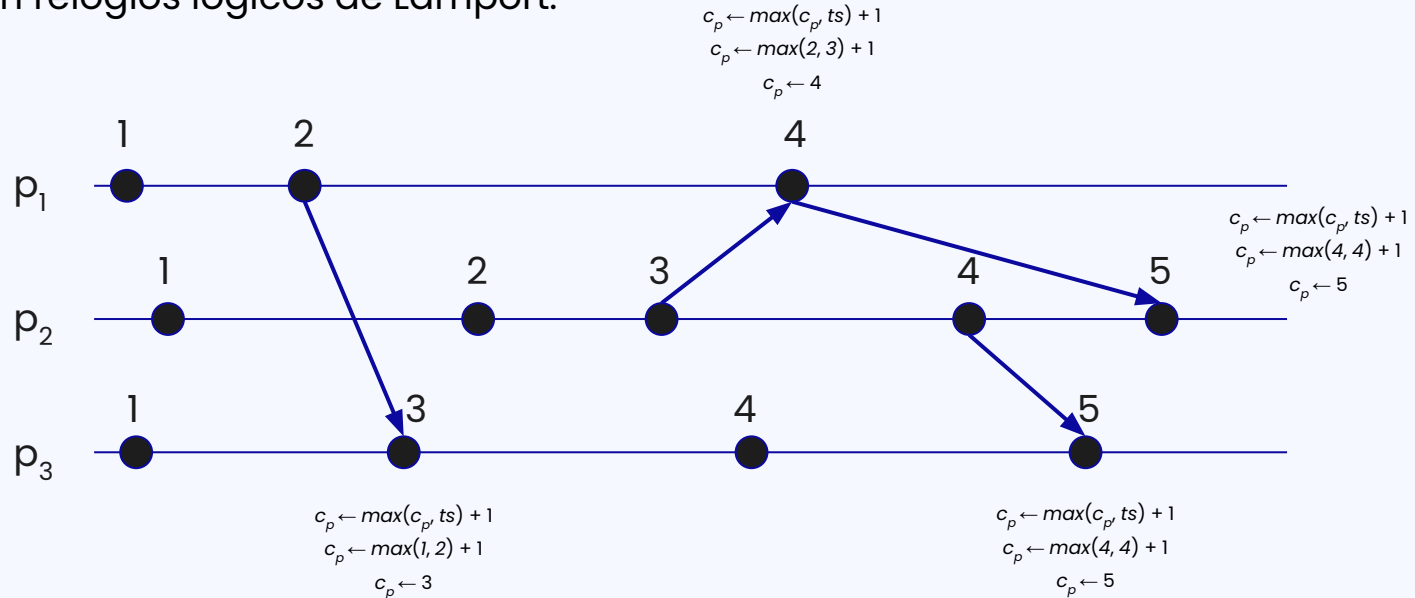


Relógio de Lamport

- Seja c_p um contador em p com valor inicialmente igual a 0.
- Se o evento é uma operação local
 - $c_p \leftarrow c_p + 1$
 - $C(e) \leftarrow c_p$
- Se o evento e é o envio de uma mensagem
 - $c_p \leftarrow c_p + 1$
 - $C(e) \leftarrow c_p$
 - $C(e)$ é enviado com a mensagem como seu *timestamp*.
- Se o evento e é a recepção de uma mensagem com *timestamp* ts
 - $c_p \leftarrow \max(c_p, ts) + 1$
 - $C(e) \leftarrow c_p$

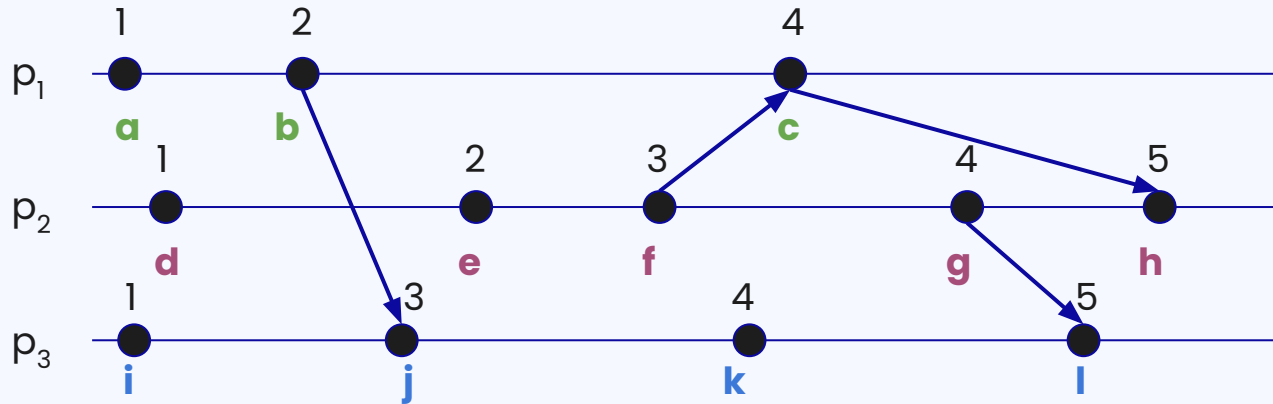
Relógios lógicos de Lamport - exemplo

- Enumere os eventos de cada processo de acordo com o que aprendido em relógios lógicos de Lamport.



Relógios lógicos de Lamport - exemplo

- Enumere os eventos de cada processo de acordo com o que aprendido em relógios lógicos de Lamport.



Uma das limitações do relógio de Lamport é que ele **não consegue identificar eventos concorrentes** que não possuem relação causal direta.

Por exemplo, considere dois processos, denominados p_1 e p_2 :

- p_1 realiza o evento **a**
- p_2 realiza o evento **b**
- p_1 e p_2 não trocaram mensagens entre esses eventos

Então a e b são **concorrentes**: não existe relação causal entre eles.

O problema é que o relógio de Lamport dá apenas um número inteiro para cada evento.
Por exemplo:

$$a = 3$$

$$b = 5$$

Isso não significa necessariamente que **a** causou **b** ou que **a** aconteceu antes de **b** de verdade.

A regra do relógio de Lamport é:

Se **a** causou **b**, então o timestamp de **a** será menor que o de **b**.

Mas o contrário não é garantido:

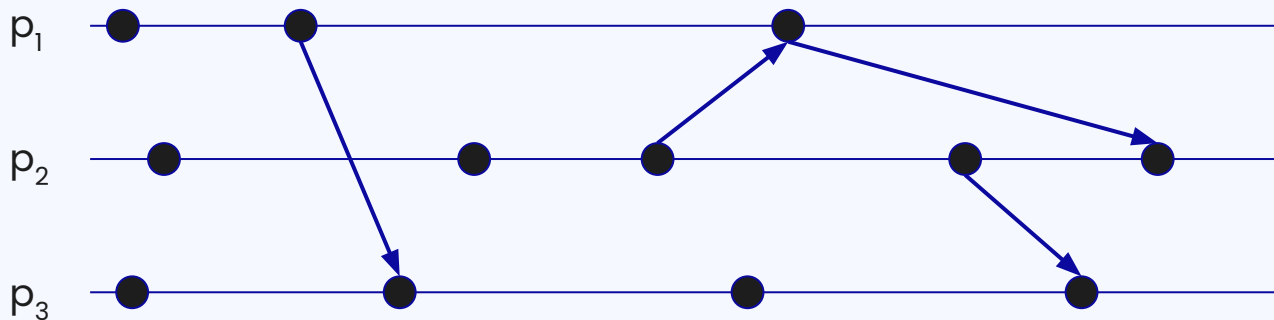
Se o timestamp de **a** é menor que o de **b**, isso não quer dizer que **a** causou **b**.

Relógios totalmente ordenados

- O problema principal em ordenar totalmente eventos é que dois ou mais eventos em diferentes processos podem ter carimbos de tempo numericamente idênticos
 - Identificadores de processo são ordenados linearmente
 - Uso de tuplas (t, i) , onde t = tempo de ocorrência e i = identidade do processo onde ocorreu.
 - Em caso de carimbo de tempo idêntico: desempate com base no identificador do processo
 - Quanto menor o identificador de processo, maior a prioridade.
- $(t_i, i) < (t_j, j)$, se e somente se $t_i < t_j$ ou $t_i = t_j$ e $i < j$.

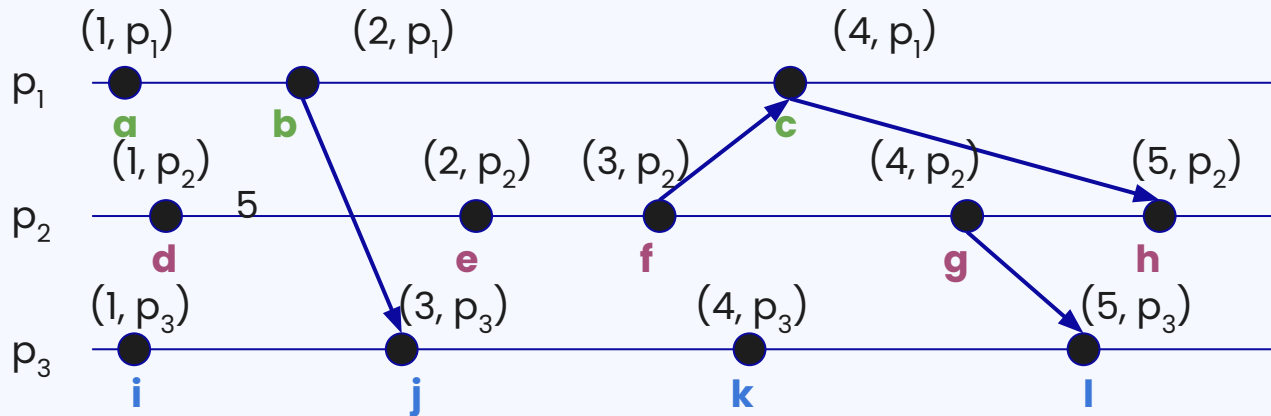
Relógios totalmente ordenados - exemplo

- Enumere os eventos de cada processo de acordo com o que aprendido em relógios totalmente ordenados.



Relógios totalmente ordenados - exemplo

- Enumere os eventos de cada processo de acordo com o que aprendido em relógios totalmente ordenados.



Relógios totalmente ordenados

- No entanto, não assegura preservação de ordem temporal dos eventos concorrentes.
- A fim de superar a deficiência dos relógios de Lamport, foram desenvolvidos os relógios vetoriais

Sincronização de relógios vetoriais

Assim como o Relógio de Lamport, o **Relógio Vetorial** também é um **relógio lógico** usado para atribuir **marcas de tempo (timestamps)** aos eventos em um sistema distribuído.

- Para resolver isso, o **relógio vetorial** utiliza **vetores de inteiros** em vez de valores inteiros únicos como timestamp.
- Se houver **N processos** no sistema, cada processo manterá um **vetor com N elementos**, representando o tempo lógico de cada processo conhecido localmente.

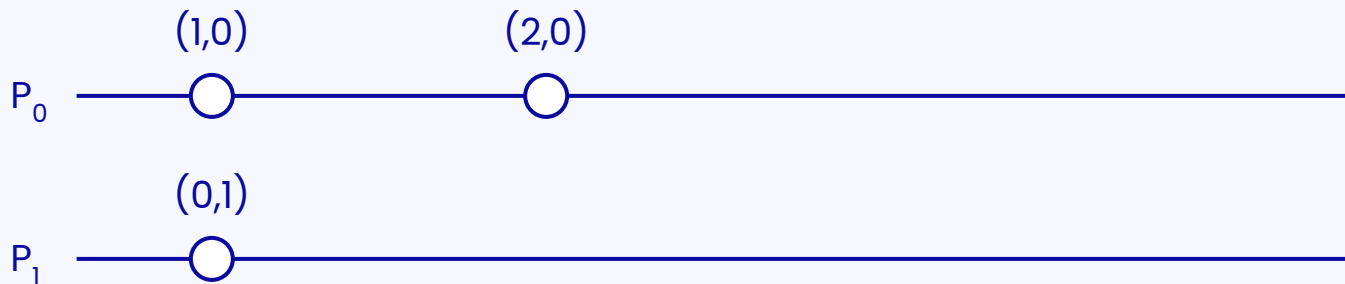
Os **relógios vetoriais** são uma extensão dos **relógios lógicos**, em que cada processo mantém não apenas um **contador dos seus próprios eventos locais**, mas também uma **visão dos contadores dos demais processos** do sistema, como um **vetor de relógios** — um para cada processo participante — que representa o seu **conhecimento sobre o progresso lógico** de todos os processos no sistema.

Regras de atualização do relógio vetorial

As regras para incrementar o timestamp são semelhantes às do relógio de Lamport, com uma modificação adicional:

1. **Antes de executar um evento**, o processo i incrementa o i -ésimo elemento de seu vetor em 1:

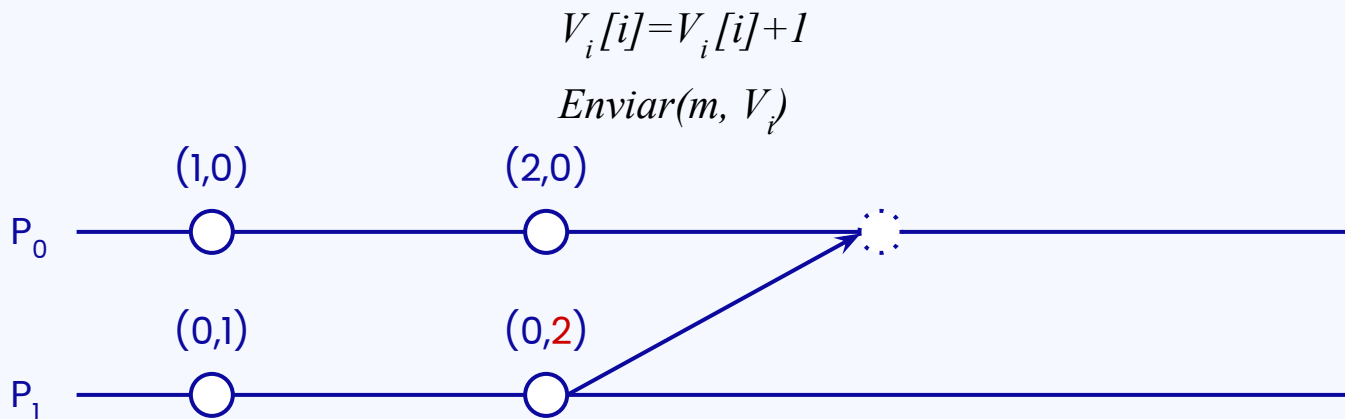
$$V_i[i] = V_i[i] + 1$$



Regras de atualização do relógio vetorial

As regras para incrementar o timestamp são semelhantes às do relógio de Lamport, com uma modificação adicional:

- **Durante um evento de envio (send)**, o processo i também incrementa o i -ésimo elemento de seu vetor em 1 e envia a mensagem junto com **seu vetor de relógio atual**:



Regras de atualização do relógio vetorial

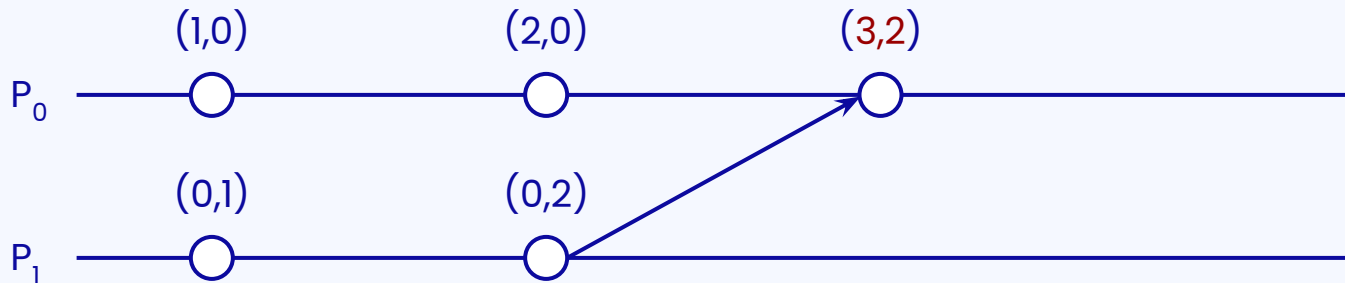
As regras para incrementar o timestamp são semelhantes às do relógio de Lamport, com uma modificação adicional:

- **Recebimento de mensagem:** Quando P_i recebe uma mensagem (m, V_j) de P_j , ele atualiza seu vetor local da seguinte forma:

$$V_i[k] \leftarrow \max(V_i[k], V_j[k]) \text{ para todo } k=1, 2, \dots, N$$

e depois incrementa seu próprio contador:

$$V_i[i] = V_i[i] + 1$$



Determinando se dois eventos são causalmente relacionados ou concorrentes:

Por exemplo, considere 3 processos P_1 , P_2 , P_3 onde cada evento tem um vetor com 3 posições:

$$V=(v1,v2,v3)$$

onde:

- v_1 é o contador que representa o conhecimento sobre os eventos de P_1 ,
- v_2 sobre P_2 ,
- v_3 sobre P_3 .

Determinando se dois eventos são causalmente relacionados ou concorrentes:

Comparando dois eventos

Agora, considere dois eventos E_1 e E_2 com vetores V_1 e V_2 .

1. Igualdade

Eles têm **o mesmo timestamp** se **todas as posições** forem iguais:

$$V_1 = V_2 \Leftrightarrow V_1[i] = V_2[i] \text{ para todo } i.$$

Exemplo:

$$(2, 1, 0) = (2, 1, 0)$$

Relação causal

Dizemos que $V_1 < V_2$ (ou que o evento 1 **aconteceu antes** de 2) se:

- Os valores de V_1 são **menores ou iguais** aos de V_2 :

$$V_1[i] \leq V_2[i] \text{ para todo } i$$

- E **pelo menos um valor é estritamente menor**:

$$V_1[j] < V_2[j] \text{ para algum } j.$$

Isso significa que o evento 1 **não tem informação mais recente que o evento 2** — ou seja, E_1 **ocorreu antes** de E_2 , ou **influenciou** E_2 .

Determinando se dois eventos são causalmente relacionados ou concorrentes:

Relação causal

Exemplo:

$$V1 = (1,0,0), V2 = (2,0,0)$$

Temos $V1 \leq V2$ e há um índice (o primeiro) em que $1 < 2$.

Logo, $E1 \rightarrow E2$ (evento 1 causou evento 2).

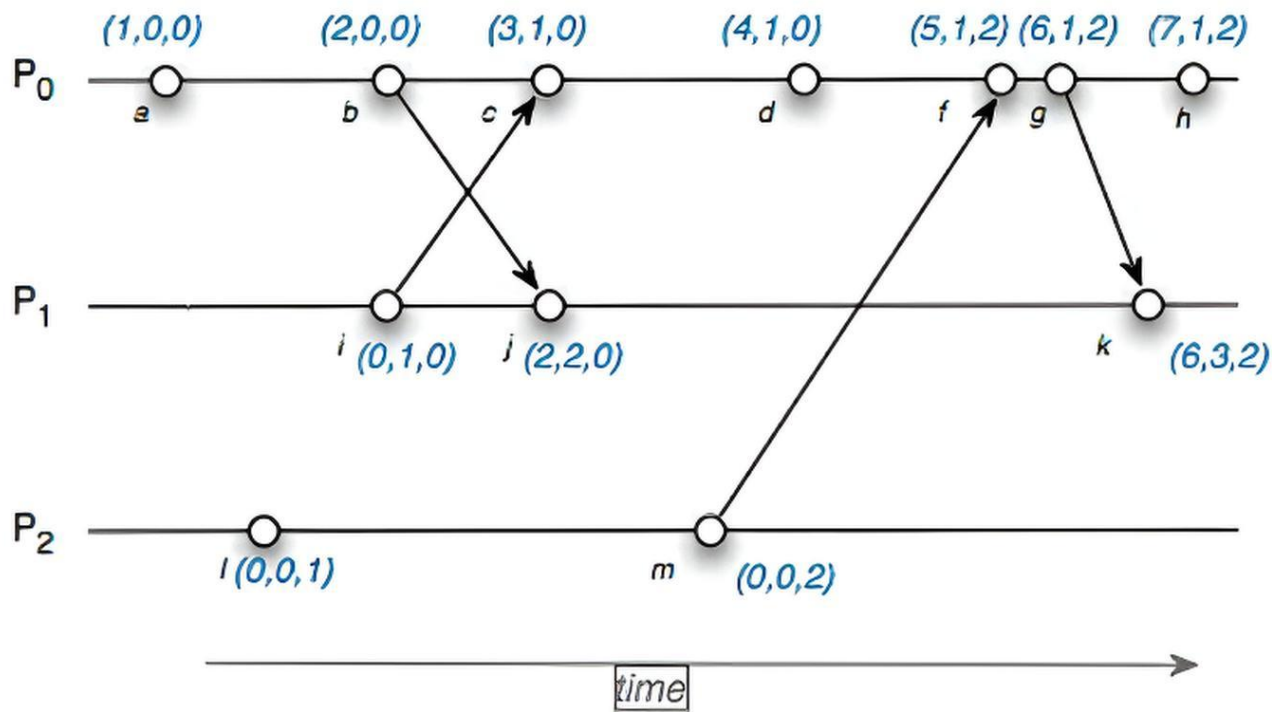
Determinando se dois eventos são causalmente relacionados ou concorrentes:

Os valores do vetor de tempo são atualizados para diferentes tipos de eventos.

Com base nisso, é possível identificar exemplos de **eventos concorrentes** e **eventos com relação causal** analisando os valores dos vetores associados a cada evento.

Exemplos de eventos causalmente relacionados

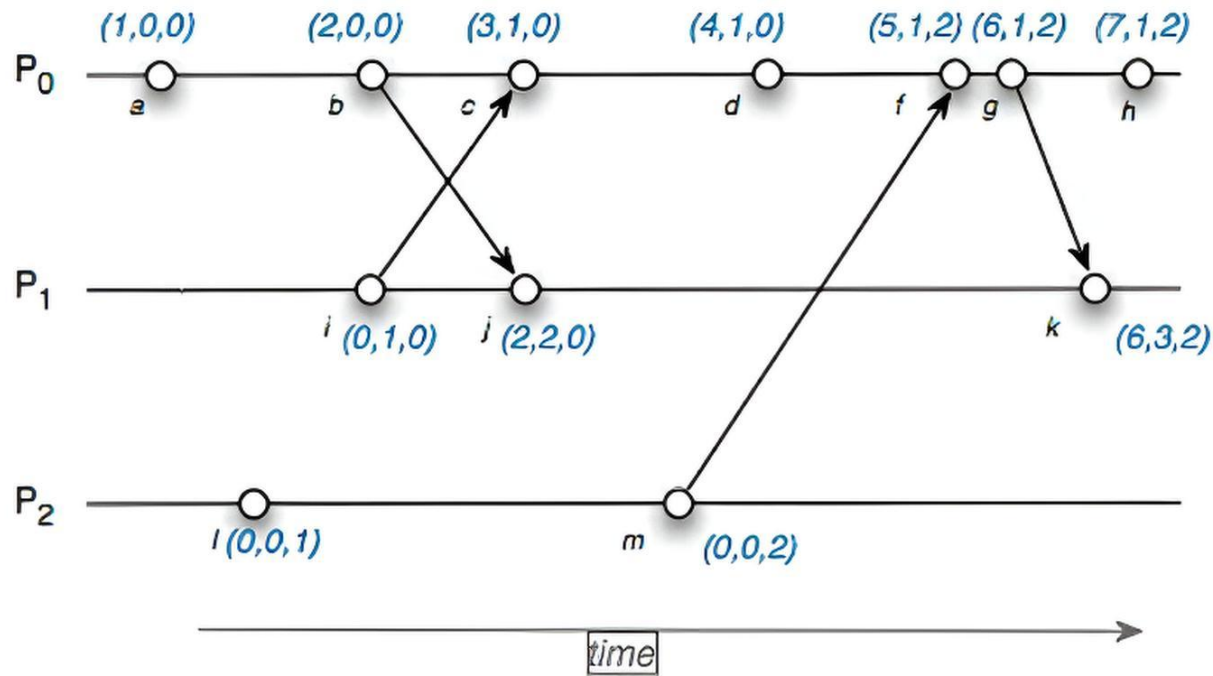
$a \rightarrow j$, pois $(1,0,0) < (2,2,0)$ $a \rightarrow b$, pois $(1,0,0) < (2,0,0)$ $m \rightarrow k$, pois $(0,0,2) < (6,3,2)$



Comparação entre timestamps vetoriais

Exemplos de eventos concorrentes

Considere os eventos **b** e **l**, com timestamps vetoriais $(2,0,0)$ e $(0,0,1)$,

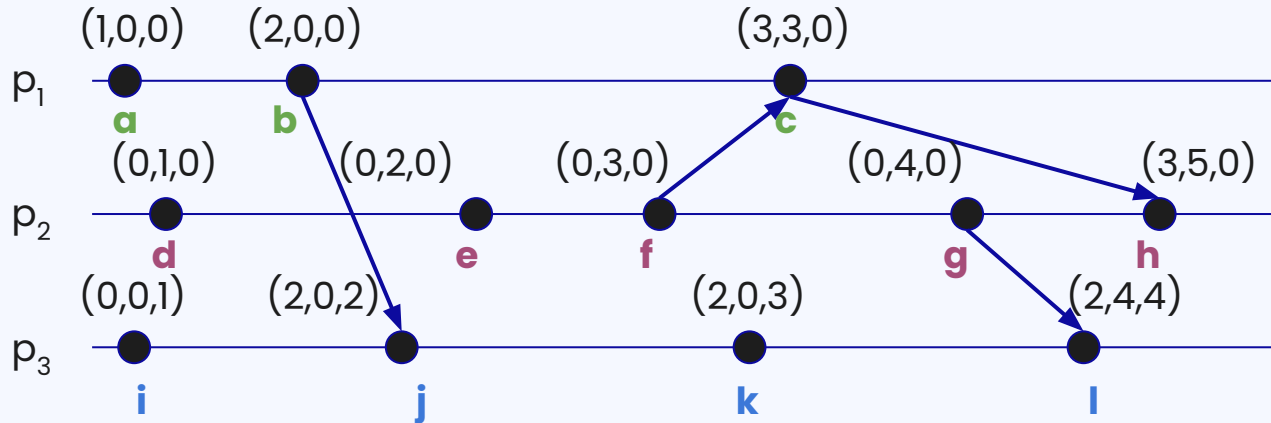


Aqui, nem $b \leq l$ nem $l \leq b$, o que significa que **não existe relação causal entre eles** — portanto, **são eventos concorrentes**.

Da mesma forma, os eventos **d** e **j** também são **concorrentes**.

Relógios vetoriais - exemplo

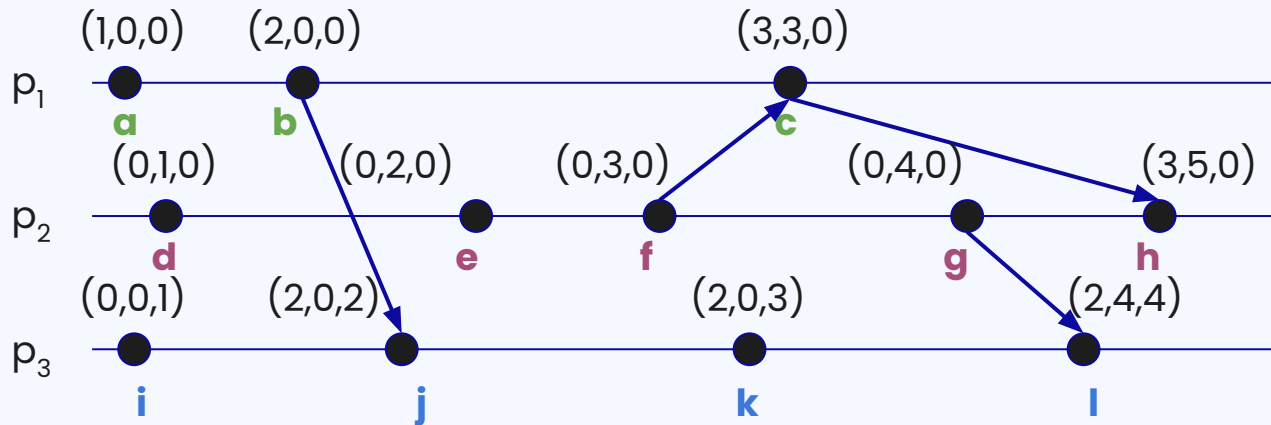
- Enumere os eventos de cada processo de acordo com o que aprendido em relógios vetoriais.



Consegue identificar os eventos causalmente relacionados?

Relógios vetoriais - exemplo

- Enumere os eventos de cada processo de acordo com o que aprendido em relógios vetoriais.



Consegue identificar os eventos concorrentes?

Observação geral

- O **relógio vetorial** é um tipo de **relógio lógico** que respeita a **causalidade** e estabelece uma **ordem parcial** entre os eventos em um sistema distribuído.
- Em comparação com o **relógio de Lamport**, o relógio vetorial consegue **identificar eventos concorrentes** de forma mais eficiente, embora **utilize mais espaço de armazenamento**, já que cada processo mantém um **vetor de N posições** (onde N é o número de processos no sistema).